InsightHub

Informe técnico del proyecto integrador · Temática B

Valentina David

Estructuras de Datos · ID 706132

Universidad Cooperativa de Colombia

Noviembre de 2026

Repositorio: github.com/ValentinaDavid156/InsightHub · etiqueta `entrega-final`

1. Descripción del sistema

InsightHub es una red social 100 % pública de preguntas y respuestas, inspirada en el ágora: no hay chats ni mensajes privados. Sus miembros entran con un `@usuario`, se hacen amigos mediante solicitud y aceptación, publican preguntas en una plaza común, responden en hilos anidados, buscan personas y publicaciones, vuelven atrás entre pantallas, leen notificaciones y reciben recomendaciones de contactos. Se usa desde la consola y los datos viven en memoria.

La priorización MoSCoW calculó 18,5 horas netas de capacidad y asignó 17,75 a las historias Must. **Se entregan completas todas las historias Must**: los 24 criterios de aceptación Must pasan, con 368 pruebas automáticas. No se implementaron las historias Should: votos, mejor respuesta, publicación destacada, conectados, búsqueda por fecha y sitio web. Al quedar fuera «conectados», la lista simple declarada en `estructuras.md` está diseñada, pero no implementada; el sistema integra las otras siete estructuras.

2. Arquitectura y diagrama de módulos

El código se organiza en tres capas. `estructuras/` contiene estructuras genéricas que no conocen el dominio; `dominio/` aplica las reglas de la temática; `main.py` solo lee, llama al dominio e imprime. Los mensajes de rechazo nacen en el dominio con el motivo y la consola los muestra, de modo que un rechazo nunca altera el estado.

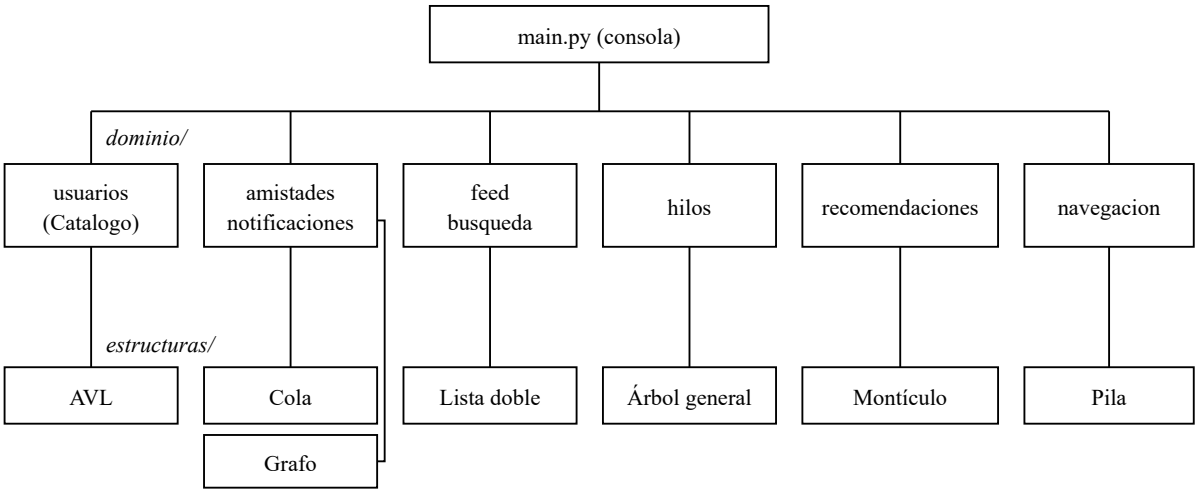


Figura 1. Módulos en tres capas. Cada módulo del dominio usa la estructura que tiene debajo; amistades usa además el grafo, recomendaciones usa grafo y montículo, y el árbol general recorre con la pila.

La integración se ve mejor en el flujo de una amistad, que atraviesa cuatro estructuras:

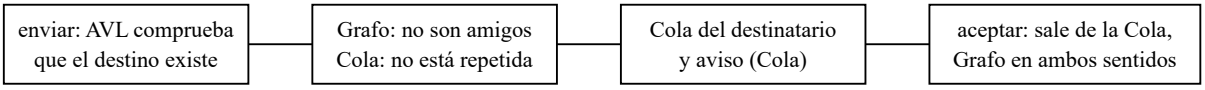


Figura 2. Flujo de una solicitud de amistad, de la consulta al índice hasta la arista del grafo.

3. Aplicación de Spec-Driven Development

Cada componente tiene `spec.md` (qué hace, con requisitos «CUANDO… EL SISTEMA DEBE» y criterios CA verificables), `plan.md` (cómo, con complejidad y alternativa descartada) y `tasks.md` (tareas en orden prueba → código). `99-sistema/spec.md` describe cómo encajan los contratos y `trazabilidad.md` enlaza cada criterio con su prueba y su código, en ambos sentidos. Las pruebas se escribieron antes del código en los avances 1 a 7, y se comprobó que fallaran. El avance 8 fue distinto: la prueba del sistema pasó de una vez, porque integraba código que ya existía.

Implementar obligó a tomar decisiones que la especificación no fijaba. Cada una quedó registrada en el historial del documento, con su motivo:

Documento	Qué cambió	Por qué
`06-grafo/spec` v1.1	De grafo dirigido de «seguimientos» a amistad no dirigida	Las historias pasaron a amistad con solicitud y aceptación
`moscow` v1.2	Must de 12,25 a 17,75 h; total de 20 a 29 h	La estimación suponía código ya preparado y no contaba las specs
`01-lista-doble/spec` v1.2	Retroceder llega al principio si faltan menos de 5	La posición se desalineaba si alguien publicaba durante la sesión
`05-avl/spec` v1.2	La arroba inicial del `@usuario` es opcional	Al implementar la entrada hubo que decidir cómo se escribe
`03-cola/spec` v1.2	Se rechaza la solicitud cruzada, con aviso	La spec no cubría que dos personas se enviaran solicitud a la vez
`99-sistema/spec` v1.2	La consola permite cambiar de `@usuario`	Con una sola sesión y datos en memoria nadie podía aceptar una solicitud
`04-arbol-general/spec` v1.2	Se rechaza la respuesta vacía	La spec no definía el texto vacío

4. Estructuras de datos

Todas están hechas con nodos o arreglos propios, sin `heapq`, `deque` ni equivalentes. Parten del código del curso (Semanas 5, 7 y 10). n es el número de elementos; las mediciones están en la sección 5.

4.1 Lista doble (plaza)

Qué es: nodos enlazados en ambos sentidos. **Implementación:** adaptada de Semana 7; cada sesión guarda un marcador propio sobre la lista y se mueve de 5 en 5. **Dónde:** la plaza (`dominio/feed.py`). **Complejidad:** publicar al frente O(1); avanzar y retroceder página O(1); buscar por palabra O(n·m). **Descartada:** arreglo, porque insertar al frente desplaza todo (O(n)); lista simple, porque retroceder exigiría recorrer desde el inicio.

4.2 Pila (historial y recorrido de hilos)

Qué es: LIFO sobre nodos, de Semana 10. **Dónde:** «atrás» (`dominio/navegacion.py`) y el recorrido del árbol general. **Complejidad:** apilar, desapilar y tope O(1). **Descartada:** lista doble con cursor, porque no se necesita «adelante».

4.3 Cola enlazada (solicitudes y notificaciones)

Qué es: FIFO con referencias al frente y al final. **Implementación:** `ColaEnlazada` de Semana 10 con `contiene`. **Dónde:** una cola de solicitudes y una de avisos por persona, y el nivel 1 del recorrido del grafo. **Complejidad:** encolar y desencolar O(1); `contiene` O(k), con k pequeño. **Descartada:** `list` con `pop(0)`, O(n); cola circular, que reserva memoria fija por persona aunque esté vacía.

4.4 Árbol general (hilos)

Qué es: árbol de ancho variable en representación primer hijo / hermano derecho (Knuth, 1997). **Implementación:** los dos enlaces del `NodoArbol` de Semana 10 cambian de significado. El recorrido apila primero el hermano y luego el hijo: como la pila saca lo último, se baja por una rama completa antes de seguir. **Dónde:** un árbol por publicación (`dominio/hilos.py`). **Complejidad:** responder, buscar y recorrer O(n). **Descartada:** `list` de hijos, por usar una estructura nativa como soporte; lista plana con id de padre, porque armar el hilo cuesta más; recursión, porque un hilo muy profundo agotaría la pila de Python.

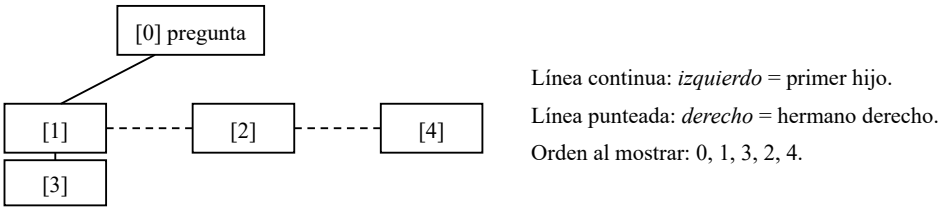


Figura 3. Un hilo con dos enlaces por nodo, sin importar cuántas respuestas tenga cada uno.

4.5 AVL (usuarios)

Qué es: árbol de búsqueda con diferencia de alturas de a lo sumo 1 en cada nodo (Adelson-Velsky y Landis, 1962). **Implementación:** parte del `NodoArbol` y de la convención de altura de Semana 10 (vacío = −1); inserción ordenada, cuatro rotaciones y búsqueda por prefijo que descarta las ramas fuera del tramo. **Dónde:** el catálogo de usuarios (`dominio/usuarios.py`). **Complejidad:** insertar y buscar O(log n); prefijo O(log n + k). **Descartada:** tabla hash, sin orden ni prefijo; árbol sin equilibrar, que con claves ordenadas degenera en una lista.

4.6 Grafo no dirigido (amistades)

Qué es: lista de adyacencia con nodos propios. **Implementación:** `agregar_amistad` valida todo antes de tocar nada, para que la amistad nunca quede en un solo sentido. Los candidatos a 2 saltos usan la cola y dos arreglos temporales. **Dónde:** `dominio/amistades.py` y `dominio/recomendaciones.py`. **Complejidad:** crear, deshacer y listar O(g), con g el número de amigos; candidatos O(V + g + E′). **Descartada:** matriz de adyacencia, O(V²) de memoria casi vacía.

4.7 Montículo máximo (recomendaciones)

Qué es: árbol completo en un arreglo (Williams, 1964). **Implementación:** recibe la función `antes(a, b)`; `construir` baja cada padre desde el último hasta la raíz, en O(m) (Floyd, 1964). **Dónde:** los 5 mejores candidatos, por amigos en común y luego alfabéticamente. **Complejidad:** insertar y extraer O(log n); máximo O(1); construir O(m). **Descartada:** ordenar los m candidatos, O(m log m).

4.8 Lista simple (conectados): no implementada

Se especificó en `08-lista-simple/` para HU-04 (inserción O(1), listar O(c) con c conectados, caducidad a los 30 segundos), pero HU-04 es Should y no entró en la capacidad.

5. Comparación empírica de decisiones

Se midió con `docs/evidencias/medir.py`: mediana de 7 repeticiones en Python 3.13. Los tiempos varían con la carga del equipo; la altura y las comparaciones son deterministas. El análisis completo y los tiempos de cada comparación están en `ANALISIS.md`.

n (claves ordenadas)	Altura AVL	Altura sin equilibrar	Insertar todo AVL	Insertar todo sin equilibrar	Buscar AVL	Buscar sin equilibrar
1 000	9	999	3,88 ms	18,95 ms	0,56 µs	53,86 µs
2 000	10	1 999	8,68 ms	75,17 ms	0,64 µs	108,28 µs
4 000	11	3 999	19,05 ms	300,68 ms	0,67 µs	213,68 µs
8 000	12	7 999	40,59 ms	1 199,12 ms	0,95 µs	454,39 µs

Al duplicar n, insertar todo sin equilibrar se multiplica por 4 y buscar por 2, como predicen $O(n^2)$ y $O(n)$. El AVL casi no cambia al buscar. Esto justifica el costo de programar las rotaciones.

m candidatos	Comparaciones montículo	Comparaciones ordenar	Montículo	Ordenar
1 000	1 978	8 631	1,96 ms	6,98 ms
10 000	18 929	120 252	22,63 ms	112,78 ms
100 000	188 347	1 531 684	234,52 ms	1 609,52 ms

El montículo hace cerca de 2 comparaciones por candidato, frente a un crecimiento m log m al ordenar. Con los 120 perfiles de la demostración la diferencia no se percibe: la decisión se justifica por el crecimiento. Para la plaza, publicar en la lista doble se mantuvo cerca de 0,2 µs, mientras que `insert(0, x)` en un arreglo pasó de 1,3 µs con 1 000 elementos a 28 µs con 100 000.

6. Pruebas y trazabilidad

La suite tiene 368 pruebas con pytest y todas pasan: AVL y usuarios 73, grafo y amistades 62, cola y notificaciones 56, consola 41, lista doble 36, árbol e hilos 36, montículo y recomendaciones 25, pila 24 y sistema 15. Cada docstring nombra el criterio que verifica.

Cubren casos extremos como estructura vacía, duplicados, las cuatro rotaciones, un hilo de 2 000 niveles y entradas inválidas. `tests/test_sistema.py` recorre por consola cada historia Must con la comunidad real de 120 perfiles.

7. Dificultades

- **Integrar lo que se probó por separado.** Al probar la consola a mano, el menú salía en desorden (1, 2, 3, 4, 7, 8, 5, 6, 0), aunque las pruebas pasaban. Se corrigió y se agregó la prueba que faltaba.
- **Una consola de una sola sesión.** Hasta integrar las amistades no se notó que, con datos en memoria y una sola persona, nadie podía aceptar una solicitud. Se resolvió con la opción «Cambiar de `@usuario`».
- **Casos que la especificación no previó.** La solicitud cruzada, la posición desalineada al paginar y los textos vacíos aparecieron al implementar y obligaron a volver a la spec.
- **Medir con ruido.** Varias corridas mostraron puntos anómalos por la carga del equipo. Por eso se usa la mediana y se reportan además medidas deterministas.
- **Capacidad.** Las horas alcanzaron para los Must; ningún Should entró, y con ello quedó sin implementar una estructura declarada.

8. Declaración de uso de inteligencia artificial

Se usó Claude Code (Anthropic, 2026) durante todo el proyecto. La IA **redactó las especificaciones, escribió todo el código y todas las pruebas**, las mediciones y este informe, por decisión de la estudiante y a partir de su código del curso. La estudiante definió la temática, tomó cada decisión de diseño eligiendo entre opciones con su justificación, recibió el detalle de cada cambio y aprobó cada commit. El registro prompt por prompt está en `BITACORA.md`. El enunciado del curso permite la IA para entender conceptos, depurar y revisar, y pide que el estudiante escriba las pruebas; este uso fue más allá y se declara aquí para que la evaluación lo tenga en cuenta. La estudiante responde por explicar el código en la sustentación.

9. Conclusiones

- Elegir la estructura por su operación dominante se reflejó en las mediciones: el AVL evita la degeneración con claves ordenadas y el montículo reduce las comparaciones cuando solo se necesitan los mejores.
- Las ventajas asintóticas importan a escala. Con los 120 perfiles de la demostración varias alternativas descartadas también funcionarían, y las mediciones lo muestran.
- Especificar antes de programar hizo visibles las decisiones, y las preguntas que quedaron abiertas aparecieron al integrar, no al probar cada estructura por separado.
- Una estructura correcta no basta: los errores más visibles estuvieron en la unión entre componentes, lo que justifica la prueba de sistema.

Referencias

Adelson-Velsky, G. M., y Landis, E. M. (1962). An algorithm for the organization of information. *Soviet Mathematics Doklady, 3*, 1259–1263.

Anthropic. (2026). *Claude Code* [Software]. https://claude.com/claude-code

Cormen, T. H., Leiserson, C. E., Rivest, R. L., y Stein, C. (2022). *Introduction to algorithms* (4.ª ed.). MIT Press.

Floyd, R. W. (1964). Algorithm 245: Treesort 3. *Communications of the ACM, 7*(12), 701.

Goodrich, M. T., Tamassia, R., y Goldwasser, M. H. (2013). *Data structures and algorithms in Python*. Wiley.

Knuth, D. E. (1997). *The art of computer programming: Vol. 1. Fundamental algorithms* (3.ª ed.). Addison-Wesley.

Python Software Foundation. (2026). *Python 3.13 documentation*. https://docs.python.org/3.13/

Williams, J. W. J. (1964). Algorithm 232: Heapsort. *Communications of the ACM, 7*(6), 347–348.